Pontifícia Universidade Católica do Rio Grande do Sul

ANTONIO DAL BEM, ARTHUR FERREIRA, LORENZO BUENO E GABRIEL BORGES

CONCORRÊNCIA EM GO

PORTO ALEGRE

2026

SUMÁRIO

1.INTRODUÇÃO.....3

2. PROBLEMA 1 CASO BASE.....3

2.1 Solução 1.....4

2.2 Solução 2.....4

2.3 Análise de Deadlock, Starvation e Fairness.....5

3.PROBLEMA 2.....5

3.1 Solução 1.....7

3.2 Solução 2.....7

3.3 Análise do Buffer.....8

4. USO DE IA.....9

5. REFERÊNCIAS.....10

Introdução

Neste trabalho exploramos alguns dos problemas que existem ao programar de forma concorrente utilizando a linguagem de programação GO: Deadlock, Starvation e Fairness. Apresentamos dois problemas diferentes, que chamamos de Philosophers e Producer-Consumer, e implementamos duas soluções para cada um deles.

O código inteiro se encontra no repositório remoto do Github, em https://github.com/LorenzoBueno/classic-concurrency-go . O código está aberto para uso geral e visualização pública,

Além das implementações das soluções aos problemas, o repositório também possui aba exclusiva para os dados coletados durante a execução do código. As métricas variam conforme o problema: em Philosophers, registramos a versão, número de filósofos, refeições concluídas, etc. Já em Producer-Consumer, registramos a implementação, capacidade do buffer, totais de itens produzidos e consumidos, etc.

Problema 1 – Jantar dos Filósofos

Na versão base, os filósofos estão em círculo e cada garfo é um channel com buffer de capacidade 1, cujo token representa o garfo livre. Todo filósofo pega primeiro o garfo da esquerda e, só depois, tenta o da direita. Inserimos uma pausa entre as duas ações para que todos segurem o garfo esquerdo antes de tentar o direito. Essa pausa torna o deadlock sempre aparente já que, sem ela, ele continuaria possível, mas dependeria das goroutines.

Após as execuções, foi confirmado que em todas o sistema entrou em deadlock e nenhum filósofo completou uma refeição. Evidenciamos também que as quatro condições de Coffman se encontram presentes no caso: há exclusão mútua já que só um filósofo pode deter cada garfo por vez; há posse e espera quando cada filósofo segura o garfo esquerdo, aguardando o direito; há não preempção pois nada retira um garfo da mão do filósofo e ele só é devolvido após a refeição; por fim, há espera circular, onde um filósofo esperar o garfo de outro forma um ciclo. Com todos segurando o garfo esquerdo, cada um espera por um recurso que só seria liberado por quem também está esperando, e o sistema para de progredir permanentemente.

A respeito de starvation, ela não se aplica. Starvation ocorre quando um processo é preterido de forma indefinida e o sistema continua progredindo. No caso base do jantar dos filósofos, nada progride a partir do momento que todos os filósofos estão bloqueados ao mesmo tempo, conceito de deadlock.

Solução 1 – Hierarquia de Recursos

Na primeira solução que implementamos, os garfos são numerados de 0 a N-1 e todo filósofo adquire primeiro o garfo de menor índice e depois o de maior índice. Para todos os filósofos isso coincide com esquerda e depois direita, onde apenas o filósofo N-1, cujos garfos são N-1 e 0, inverte a ordem e pega primeiro o garfo 0. Com isso, a simetria da versão base é quebrada.

Aqui, portanto, o deadlock não ocorre, já que a hierarquia de recursos elimina a espera circular, uma das condições de Coffman. A condição é eliminada pois, como os garfos são adquiridos em ordem crescente, a cadeia de espera nunca volta para o ponto de partida.

A starvation não foi observada e não é impedida diretamente pela solução. A hierarquia não garante que um filósofo específico consiga os garfos em certo tempo limitado. Nos testes que realizamos, todos filósofos completaram as refeições. Porém, a nossa quantidade de execuções não determina que a starvation seja impossível.

Como todos os filósofos executam exatamente R iterações (refeições), o número de refeições é igual para todos e não diferencia os filósofos. Avaliamos a igualdade pelo tempo médio de espera pelos garfos. Para isso, utilizamos o índice de Jain, $J = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$, que varia de 1/n, que seria a máxima desigualdade, a 1, distribuição perfeita. Os resultados foram J ≈ 0,958 (N=5), 0,957 (N=10) e 0,948 (N=50), valores altos quando comparados ao pior caso possível (0,20; 0,10 e 0,02, respectivamente).

Concluímos que a hierarquia de recursos é razoavelmente justa nesse cenário, sem starvation observado, mas distribui as esperas de forma não uniforme, e essa desigualdade aumenta com o número de filósofos.

Solução 2:

Na estratégia limit, usamos um channel com capacidade N−1 para controlar quantos filósofos podem tentar pegar os garfos ao mesmo tempo. Antes de tentar adquirir os garfos, cada filósofo precisa ocupar uma dessas vagas. Depois, ele pega os dois garfos adjacentes, come, libera os garfos e só então libera a vaga para outro filósofo. Assim, no máximo N−1 filósofos disputam os garfos simultaneamente.

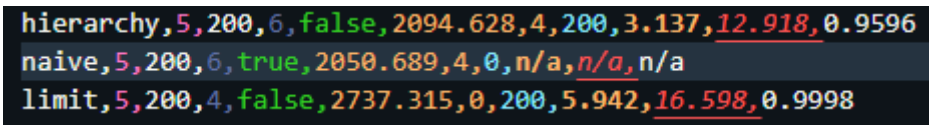
Essa restrição evita a situação em que todos os filósofos seguram o garfo da esquerda e ficam esperando pelo da direita. Como pelo menos um filósofo não entra na disputa a cada rodada, não é possível que todos os garfos fiquem simultaneamente retidos por filósofos que aguardam outro garfo. Isso impede a espera circular que causa o deadlock na versão base. Nas execuções

registradas, a estratégia terminou sem deadlock e todos os filósofos completaram as refeições previstas.

A limitação, porém, não garante starvation freedom. Ela reduz a concorrência, mas não determina qual filósofo conseguirá uma vaga ou um garfo primeiro; além disso, a ordem de atendimento de goroutines esperando em channels não é uma garantia de justiça estrita. Portanto, um filósofo poderia, em princípio, esperar mais que os demais. Nos dados coletados para N=5 e R=200, isso não foi observado: todos os filósofos completaram 200 refeições em cada uma das três repetições.

Para comparar a distribuição das esperas, usamos o índice de Jain calculado sobre o tempo médio de espera pelos garfos. Nas três execuções, os valores foram aproximadamente 0,9998, 0,9995 e 0,9990, próximos de 1, que representa uma distribuição uniforme. Isso indica que nesse cenário, os tempos médios de espera ficaram bastante equilibrados entre os filósofos. O resultado é uma observação dessas execuções, não uma garantia de que a estratégia sempre será igualmente justa. Além disso, a métrica de espera considera o tempo aguardando a aquisição dos garfos, mas não o tempo aguardando uma vaga no limite de N−1 filósofos.

Análise de Deadlock, Starvation e Fairness



Em termos de análise de desempenho e justiça de acesso aos recursos, a linha de saída “hierarchy,5,200,6,false,2094.628,4,200,3.137,12.918,0.9596” representa um caso em que a estratégia hierarchy foi executada com 5 filósofos, 200 refeições por filósofo, na repetição 6, sem deadlock (false), com tempo total de 2094,628 ms, para o filósofo de índice 4, que realizou 200 refeições, com espera média de 3,137 ms, espera máxima de 12,918 ms e índice de fairness de Jain igual a 0,9596. Nesse contexto, o primeiro campo indica a estratégia adotada; o segundo, o número de filósofos; o terceiro, o número de iterações ou refeições; o quarto, a repetição da execução; o quinto, se houve deadlock; o sexto, o tempo total em milissegundos; o sétimo, o identificador do filósofo; o oitavo, o número de refeições concluídas por ele; o nono, o tempo médio de espera; o décimo, o tempo máximo de espera; e o último, o índice de fairness de Jain, que varia de 0 a 1, sendo 1.0 a situação ideal de equidade. Assim, o valor de 0,9596 indica que a distribuição de espera foi muito próxima da ideal, ou seja, os filósofos tiveram tempos de espera relativamente equilibrados.

No que se refere ao deadlock e à starvation, a estratégia naive é a que mais claramente apresenta deadlock: em todas as execuções registradas, o

campo deadlock aparece como true, indicando que os filósofos ficam presos em espera circular ao tentar adquirir os garfos da esquerda e da direita, sem conseguir prosseguir. Esse comportamento está de acordo com o conceito de deadlock estudado em aula: há exclusão mútua, posse de recursos e espera circular, de modo que nenhum filósofo consegue concluir a execução. Já as estratégias hierarchy e limit não apresentam deadlock nos dados analisados; em ambas, o campo deadlock é false, e todos os filósofos conseguem completar as suas refeições. A starvation, por sua vez, é menos evidente nos resultados: a hierarchy apresenta tempos de espera relativamente uniformes, enquanto a limit exibe uma equidade ainda mais alta, com índice de Jain próximo de 1. Isso sugere que, para os casos testados, não houve situação de um filósofo "ignorado" indefinidamente, mas sim uma distribuição de acesso aos garfos de forma balanceada. Em comparação, a naive não deve ser avaliada como justa em sentido prático, porque o sistema não conclui seu progresso; o valor 1.0000 no índice de Jain aparece apenas por consequência de esperas nulas em uma execução interrompida, e não representa justiça real no sistema. Dessa forma, a análise dos dados mostra que a hierarchy e, sobretudo, a limit promovem maior justiça e progresso, enquanto a naive falha por deadlock e não representa uma execução funcional.

Problema 2 - Produtor/Consumidor com Buffer Limitado:

O problema do produtor/consumidor com buffer limitado é um dos problemas clássicos de sincronização em programação concorrente. Nele, P produtores geram itens e C consumidores os processam, e os dois grupos se comunicam por um buffer compartilhado de capacidade K. O buffer é a única ponte entre eles, e isso exige coordenação: quando está cheio, os produtores devem esperar até surgir espaço, e quando está vazio, os consumidores devem esperar até chegar um item. Além disso, como vários produtores e consumidores acessam o mesmo buffer ao mesmo tempo, é preciso garantir que nenhum item seja perdido nem entregue a mais de um consumidor.

Neste trabalho implementamos o problema em Go de duas maneiras. Na primeira, o buffer é um channel com capacidade K, e usamos a semântica de bloqueio do próprio Go. Na segunda, o buffer é uma estrutura de dados compartilhada, protegida explicitamente por semáforos de contagem e por exclusão mútua, reproduzindo o algoritmo clássico. As duas versões foram comparadas quanto a deadlock, starvation, encerramento ordenado e efeito da capacidade K sobre o desempenho.

Solução 1 — Produtor/Consumidor com Channel:

Para resolver este problema, implementamos um buffer limitado usando channel com capacidade K. Usamos várias goroutines para representar os produtores e consumidores, e o próprio channel bloqueia os produtores quando o buffer está cheio e os consumidores quando está vazio. Também usamos select com time.After em um dos consumidores para registrar timeouts e continuar aguardando novos itens. Para encerrar corretamente, esperamos todos os produtores terminarem, fechamos o channel e deixamos os consumidores processarem os itens restantes. Coletamos métricas de throughput, quantidade de itens por consumidor e ocupação média aproximada do buffer para K igual a 1, 10 e 100. Nas execuções realizadas, os 4.000 itens produzidos foram consumidos, e os testes com o detector de corrida não apontaram data races.

Solução 2 - Versão com semáforos:

Na versão com semáforos do problema produtor/consumidor, o buffer compartilhado de capacidade K é controlado por dois semáforos de contagem, que coordenam produtores e consumidores. Um conta as posições livres e faz o produtor esperar quando o buffer está cheio. O outro conta os itens disponíveis e faz o consumidor esperar quando está vazio. Com vários produtores e consumidores, só contar não basta, pois dois produtores poderiam escrever na mesma posição e dois consumidores retirar o mesmo item. Por isso, o acesso às posições de escrita e de leitura é protegido por exclusão mútua, o que garante que cada posição seja usada por um só processo de cada vez e que cada item seja entregue a um único consumidor.

O encerramento é ordenado. Quando os produtores terminam, são colocados no buffer marcadores de fim, um para cada consumidor. Como o buffer segue a ordem de chegada, cada consumidor só encontra o marcador depois de retirar todos os itens que sobraram. Assim, nenhum item é perdido, nenhum consumidor fica bloqueado para sempre e não há deadlock. Ao final, o programa confere se o total produzido é igual ao total consumido. Para o consumo com timeout, um dos consumidores espera por um item por um tempo limitado. Se nada chega nesse intervalo, ele registra o evento e volta a esperar, em vez de ficar parado indefinidamente.

Por fim, coletamos dados de execução variando a capacidade do buffer (K = 1, 10 e 100), com o número de produtores e consumidores fixo. Isso permite

observar como o tamanho do buffer afeta o desacoplamento entre produtores e consumidores. Um buffer maior absorve melhor as diferenças de velocidade entre os dois lados, e um buffer pequeno obriga os dois a trabalharem quase em sincronia.

Análise do Buffer:

Versão	K	Throughput	Ocupação média do buffer	Itens por consumidor (C0 / C1 / C2 / C3)	Timeouts
Channel	1	382.431	0,49	1077 / 958 / 924 / 1040	0
Channel	10	636.280	4,83	919 / 1034 / 1132 / 914	0
Channel	100	663.048	44,87	840 / 1021 / 939 / 1200	0
Semáforo	1	1.252.122	0,50	979 / 995 / 1013 / 1013	0
Semáforo	10	1.593.983	5,56	752 / 1075 / 1107 / 1066	0
Semáforo	100	2.026.981	46,39	759 / 1093 / 996 / 1152	0

Não há deadlock em nenhuma das versões. Na versão com channel, produtores e consumidores só esperam uns pelos outros, sem espera circular. Na versão com semáforos, a ordem de aquisição é sempre a mesma (primeiro o semáforo, depois o mutex, que é liberado logo em seguida), então ninguém espera por um semáforo segurando um mutex, o que quebra a condição de posse e espera de Coffman. Starvation também não ocorreu: em todas as execuções, todos os consumidores processaram itens, e o menor valor foi 704 de 4000 (o ideal seria 1000). A distribuição é desigual, mas por causa do escalonamento das goroutines e não por um consumidor ser preterido. O Go enfileira as goroutines bloqueadas, mas isso é comportamento do runtime e não uma garantia formal de justiça.

Sobre o efeito do buffer, nas duas versões, o throughput sobe com K, porque um buffer maior absorve as diferenças de velocidade entre produtores e consumidores e os desacopla. Com K=1 os dois lados trabalham quase em sincronia. Na média de 3 repetições, o channel foi de 382 mil para 636 mil e 663 mil itens/s (K=1, 10 e 100), e os semáforos de 1,25 milhão para 1,59 milhão e 2,03 milhões. No channel o ganho praticamente estabiliza depois de K=10. A

ocupação média ficou perto de K/2 nas duas versões, o que indica ritmos parecidos entre produtores e consumidores. As versões se comportaram de forma equivalente, com diferença só na medição: a ocupação é exata nos semáforos (contador) e aproximada no channel (amostras de `len(buffer)`). As duas foram medidas em máquinas diferentes, então só a tendência por K pode ser comparada, não os valores absolutos de throughput.

Por fim, sobre o encerramento ordenado, na versão com channel, o coordenador espera todos os produtores terminarem e fecha o channel. Os consumidores continuam recebendo até esvaziá-lo e então encerram, sem perder itens. Na versão com semáforos, quando os produtores terminam, o coordenador coloca no buffer um marcador de fim por consumidor. Como o buffer segue a ordem de chegada, cada consumidor só encontra o marcador depois de retirar os itens restantes, e cada um retira um marcador e encerra. Nas duas versões o programa confere ao final que o total produzido é igual ao total consumido.

Uso de IA:

Utilizamos o Claude como ferramenta de apoio na geração de código. Ele ajudou principalmente no desenvolvimento do código, nos testes e no script de coleta de dados. O código gerado foi revisado, testado e executado por nós, inclusive com o detector de data race.

Também utilizamos o Claude como apoio na redação e revisão de trechos do relatório, que foram conferidos por nós com base nos dados coletados.

Referências:

- ANTHROPIC. Claude. Disponível em: https://claude.ai/. Acesso em: 13 out. 2026.

- GOOGLE. The Go Programming Language. Disponível em: https://go.dev/. Acesso em: 13 out. 2026.

- GITHUB. GitHub. Disponível em: https://github.com/. Acesso em: 13 out. 2026.